# The Future of Ruby Documentation

Stan Lo

# About me

- Ruby committer, RDoc maintainer
- Shopify — Ruby & Rails Infrastructure team
- I work on developer tools: IRB, Ruby LSP, RDoc, debug.gem

# This past year, I've been working on RDoc.

I want to share what's changed, what's in progress, and how I'm thinking about what comes next.

**01** **Caught up**
New theme, live preview, Prism parser

**02** **Getting there**
Markdown support, RBS type signatures

**03** **Getting ahead**
AI and documentation

# What RDoc does

- Ruby's default documentation tool — ships with Ruby since 1.8
- Parses Ruby source, C extensions, and markup files
- Generates HTML docs and `ri` terminal docs
- Supported markup: RDoc, Markdown, RD, TomDoc
- Powers **docs.ruby-lang.org**

# Some context

RDoc has been Ruby's documentation tool for 20+ years. It works. But a lot has changed since it was written.

- Developers expect dark mode, mobile support, live preview
- Markdown became the standard way to write documentation
- Ruby syntax grew beyond what the parser was built for
- Type annotations exist now — documentation doesn't show them

01

# Caught up

# Before: Darkfish

[ Insert Darkfish screenshot ]

No dark mode · Limited mobile · Dated design · Basic search · Embedded fonts

# Aliki

[ Insert Aliki screenshot ]

- Three-column layout with TOC
- Fuzzy search with type badges
- Method signature cards
- Dark mode with system detection
- Mobile-first responsive
- C & bash syntax highlighting

(Named after my cat.)

# Aliki: Search

[ Insert search UI screenshot ]

- Tiered scoring: exact > prefix > substring > fuzzy
- Namespace-aware: `String#` filters to String methods
- Type badges: classes · modules · methods · pages

# Aliki: Dark mode & mobile

[ Insert light/dark + mobile screenshots ]

# `rdoc --server`

[ Insert live-reload demo ]

- Live-reloading documentation preview
- Zero dependencies — uses Ruby's `TCPServer`
- Incremental re-parsing — only changed files
- Also: `make html-server` in ruby/ruby

# Prism migration — tompng

I want to specifically highlight tompng's (Tomoya Ishida) work. He rewrote **3 major subsystems**:

**Ruby parser**: Ripper → Prism AST visitor

20-month effort · Fixes 7+ tracked issues · Modern syntax support

**Comment directive parser**: `:call-seq:` , `:nodoc:` , etc.

Fixed double-parsing architecture

**Inline formatting engine**

Replaced string macros with structured parser

43 PRs total. Invisible to users, foundational for everything else.

# What "caught up" means

| Reading | Authoring | Parsing |
| --- | --- | --- |
| Aliki theme | Server mode | Prism |

These are things modern documentation tools should have. Now RDoc has them too.

02

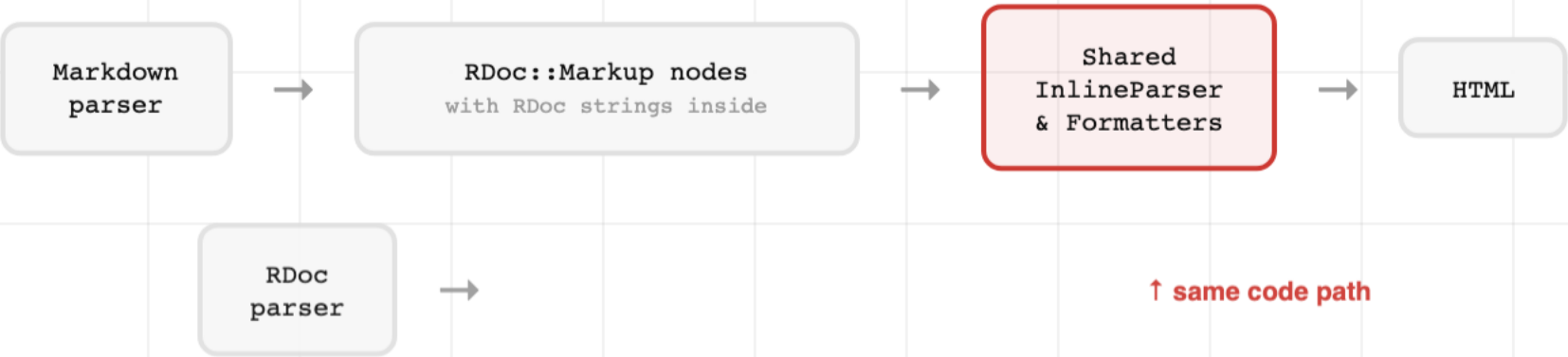
# Getting there

# Why Markdown?

- Markdown is the industry standard
- RDoc markup is Ruby-specific — even within the community, it's not widely known
- Many contributors already write Markdown in RDoc comments without realizing it doesn't render correctly

If you can write a GitHub README, you can contribute to Ruby documentation.

# Why it took so long




Markdown was added ~2012 by reusing RDoc's pipeline. **Every Markdown fix is a two-format fix.**

# Example: `~~strikethrough~~`

```
# Markdown parser outputs:
~~text~~ → "<del>text</del>"  # plain string!

# Shared InlineParser receives that string
# ...but didn't recognize <del>
# Strikethrough was silently broken for years

# Fix: modify the shared InlineParser
#   → must verify RDoc markup still works too
```

> *Every Markdown fix is a two-format fix.*

# What's improved

- `~~Strikethrough~~` aligned with GFM spec
- GitHub-style heading anchors ( `#heading-name` )
- Table parsing: inline markdown, incomplete rows
- Bash/shell syntax highlighting
- Backtick quoting in RDoc markup too
- `break_on_newline` enabled by default

NEW GFM spec comparison test suite

# RDoc markup vs Markdown

## RDoc markup

```
= Heading
*bold*  _italic_  +code+
{link text}[url]
* bullet item
```

## Markdown

```
# Heading
**bold**  *italic*  `code`
[link text](url)
- bullet item
```

# Migration plan

1. Get Markdown to GFM-level quality ← in progress
2. Build migration tooling (RDoc markup → Markdown)
3. Migrate ruby-core documentation
4. Deprecate RDoc markup? (open question)

The URL already changed: `standard_library_rdoc.html` → `standard_library_md.html` in Ruby 3.4+

# RBS type signatures in documentation

```
# Inline RBS annotation
#: (String name, ?Integer age) -> User
def create_user(name, age = nil)
  # ...
end
```

RDoc extracts `#:` annotations and displays type signatures in HTML and `ri`. Type names are linked to their documentation.

# RBS in HTML output

[ Insert HTML output screenshot with type signatures ]

Type names link to documentation · Signature cards with types · Powered by RBS gem ≥ 4.0

# RBS in `ri`

[ Insert ri terminal output with type signatures ]

# Can RBS replace `:call-seq:` ?

## :call-seq:

```
# :call-seq:
#   readlines(sep=$/)      -> array
#   readlines(limit)       -> array
#   readlines(sep, limit) -> array
```

## RBS #:

```
#: (?String sep) -> Array[String]
#: (Integer limit) -> Array[String]
#: (String, Integer) -> Array[String]
```

The overlap is significant. For a language that doesn't want typing, writing type signatures for documentation is a big shift. But I think it's the right direction — structured signatures are better for both humans and tools.

# The gap: default values

## :call-seq:

```
commercial(cwyear, cweek=1, cwday=1, sg=nil)
```

Shows actual default values

## RBS:

```
(Integer, ?Integer, ?Integer, ?Symbol?) -> Date
```

Only says "optional" — default is lost

> *If we want to migrate, RBS needs default value support.*

# Where this stands

- I'm aiming to ship RBS display on **docs.ruby-lang.org**
- The call-seq migration depends on RBS supporting default values — that's not solved yet
- This needs community discussion, not just my opinion

But the direction seems clear: structured, machine-readable signatures are better than free-form text.

03

# Getting ahead

# llms.txt: I prototyped it

Then I looked at the data:

- **10% adoption** across 300K domains
- **Zero measurable impact** on AI citations
- ML model improved when llms.txt was *removed*
- Zero bot visits from GPTBot, ClaudeBot, etc.
- No major LLM provider officially supports it

The trade-off is generation time and code complexity for every RDoc user, for a benefit that doesn't exist yet. So I decided not to pursue it — at least for now.

# What I think actually helps

Instead of generating special files for LLMs, I'd rather make the documentation itself better:

- **Markdown** — any tool can parse it, humans and machines alike
- **RBS types** — structured data instead of free-form text
- **Clean architecture** — makes it easier to build things we haven't thought of yet

On the tooling side, I've been building `github.com/st0012/ruby-skills` — Claude Code plugins for Ruby development. No RDoc-specific skills yet, but that's coming once the Markdown work is further along.

# How I think about this

- Check the data before building a feature — llms.txt looked promising until I looked at the numbers
- Invest in foundations that serve multiple purposes — Markdown, types, and clean architecture all make docs better for everyone, including future tools
- As a maintainer, I'd rather ship fewer things that are solid than chase every trend

# Summary

| CAUGHT UP | GETTING THERE | GETTING AHEAD |
|---|---|---|
| **Aliki theme** | Markdown / GFM | Evaluate with data |
| **Server mode** | RBS type signatures | Build foundations |
| **Prism parser** | call-seq → RBS? | Serve all consumers |

# Try it today

- Contribute to Ruby docs — **it's Markdown now!**
- Try `rdoc --server` for your own projects

`github.com/ruby/rdoc`

`docs.ruby-lang.org`

`github.com/st0012/ruby-skills`

# Thank you!

tompng, the Shopify Ruby DX team,
and the Ruby committers who reviewed all those PRs

@st0012